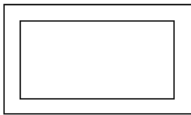
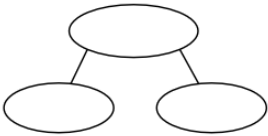
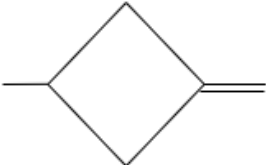
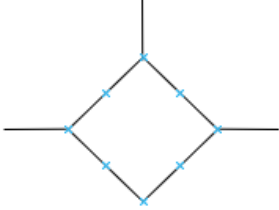
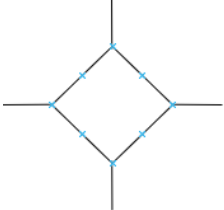
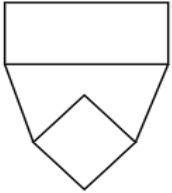
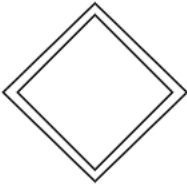
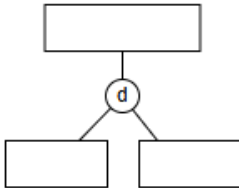
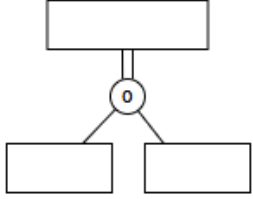


CSE370 Mid Review

!! Review AFTER you have completed the slides in buX !!

ER/EER

Things to keep in mind:

Strong entity 	Weak entity 	Simple Attributes 	Partial Attribute 
Composite Attribute 	Multivalued Attribute 	Composite + Multivalued 	Key Attribute 
Derived Attribute 	Binary Relationship 	Ternary Relationship 	N-ary Relationship(N>3) 
Recursive Relationship 	Identifying Relationship 	Disjoint Specialization  Disjoint $\Rightarrow$ d Optional Participation $\Rightarrow$	Overlapping Specialization  Overlapping $\Rightarrow$ o Total Participation $\Rightarrow$
Cardinality Ratio 1:1 $\Rightarrow$ One to One, 1:N $\Rightarrow$ One to Many, M:N $\Rightarrow$ Many to Many [Must Use different letters for M:N] M:N:K: Many to Many to Many [Ternary]		Existence Dependency Single Line for Optional Participation Double Line for Total Participation NOTE: If Cardinality and Participation are not Clearly mentioned OR can't be inferred from Question, Make Logical Assumption	

Database Constraints

Things to keep in mind:

- Every Table must have one Primary Key (PK).
- The Primary Key can be composed of one or more attributes.
- The PK is denoted with an underline.
- Foreign Keys are not native to their own table but are referencing (with an arrow) to their native/ referenced table.
- Like this: Foreign Key (FK) -----> Primary Key (PK)
- Insert, Update operation can cause all 4 constraint violation
- Delete operation can cause only FK constraint

How to **identity** Database Constraints:

Constraints	Description
1. Key Constraints	No duplicate Primary Key Value
2. Entity Integrity Constraints	No null value for Primary Key
3. Referential Integrity Constraints (FK Constraint)	Inserted new Foreign Key (FK) must match from Referenced Key (PK) (IF the FK is part of PK, else can be null)
4. Domain Constraints	DataType should match, set rules with check

How to **enforce** Database Constraints:

Constraints	Prevention / Solution / Enforce Constraint
1. Key Constraints	Set the attribute as Primary Key
2. Entity Integrity Constraints	Set the attribute as Primary Key
3. Referential Integrity Constrains (FK Constraint)	On Insert: Set proper Foreign Key with Primary key. On Update/Delete: Restrict -> operation does not run Cascade -> Delete rows where it matches Set Null -> Set Null where it matches
4. Domain Constraints	Set the attribute with proper DataType, set rules

Schema Mapping

Things to keep in mind:

Step	Mapping Process	Details
Step 1	Mapping of Regular/Strong Entity Types	Primary Key should be underlined
Step 2	Mapping of Weak Entity Types	Both Partial Key and Foreign Key of Strong Entity should be underlined
Step 3	Mapping of Binary 1:1 Relationship Types	FK placed in the relation with Total Participation, otherwise any relation is fine. FK points towards referenced PK (not underlined, use an arrowhead)
Step 4	Mapping of Binary 1:N Relationship Types	FK placed in the relation on the N side. FK points towards referenced PK (not underlined, use an arrowhead)
Step 5	Mapping of Binary M:N Relationship Types	Create a separate relation with both participating entities' Keys as a composite key. Both keys should be underlined, and both are FKs (they point to their respective PKs).
Step 6	Mapping of Multivalued Attributes	Create a separate relation with the primary key of that entity and the multivalued attribute. Both the key and the attribute should be underlined. FK points towards PK.
Step 7	Mapping of N-ary Relationship Types	Create a separate relation where the PK will contain all the participating entities' keys. All keys should be underlined and FKs point toward PK.

Superclass / Subclass schema mapping:

Multiple Relations	8A: Multiple relations - superclass and subclasses 8B: Multiple relations - subclass Only
Single Relation	8C: Single relation - with one type attribute 8D: Single relation - with multiple type attribute (flags for each subclass)

8A: Can be applied **for all types** (total or optional, disjoint or overlapping)

8B: Can be applied if the specialization is **Total Only**

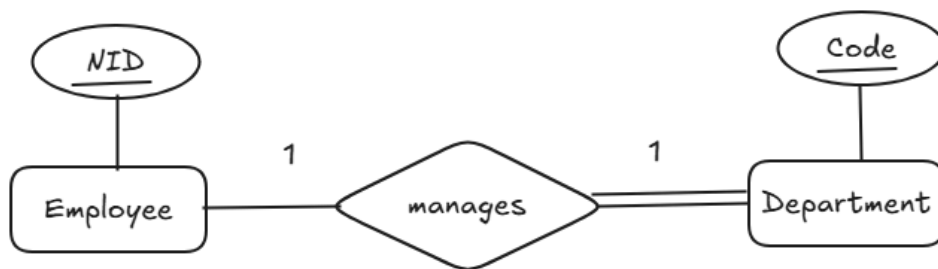
8C: Can be applied if the specialization is **Disjoint Only**

8D: Can be applied **for all types** (total or optional, disjoint or overlapping)

SCHEMA MAPPING EXAMPLES [Pictures Attached]

Binary Relationships

- 1:1 Relationship Mapping

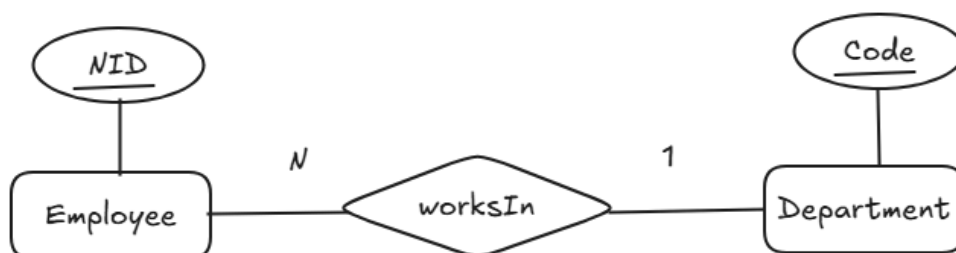


Schema Diagram



Note:
FK approach,
FK put in total side,
NID not underlined in Department

- 1:N Relationship Mapping

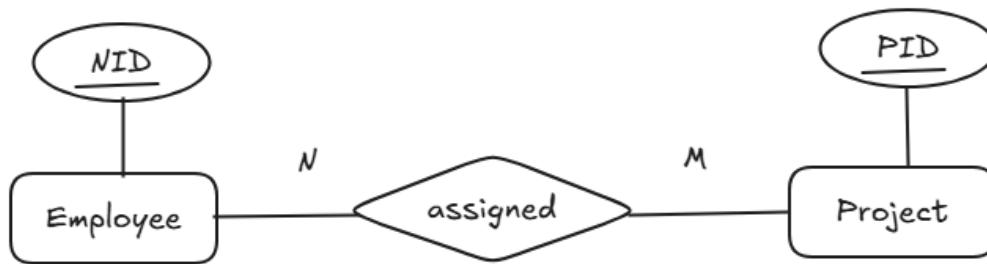


Schema Diagram

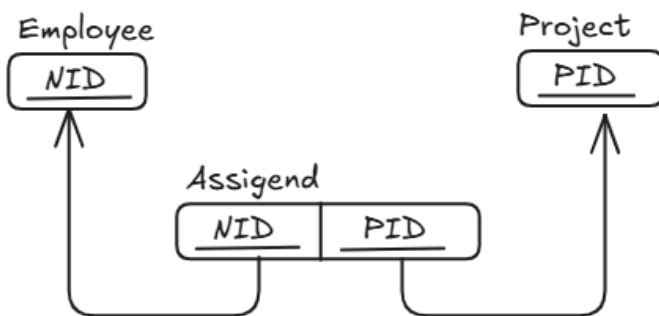


Note:
FK approach,
FK put in N side,
Code not underlined in Employee

- M:N Relationship Mapping

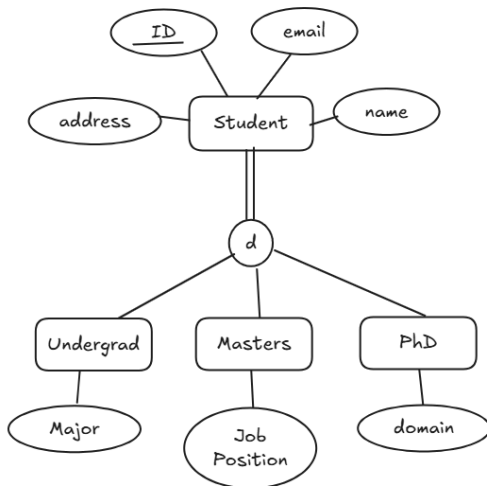


Schema Diagram

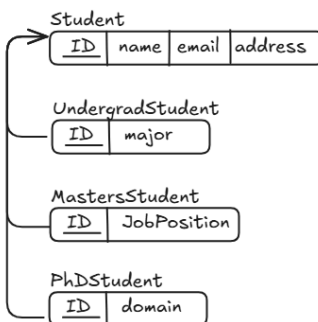


Note:
Cross Referenced Approach,
Create Separate Relation,
Both FK are underlined

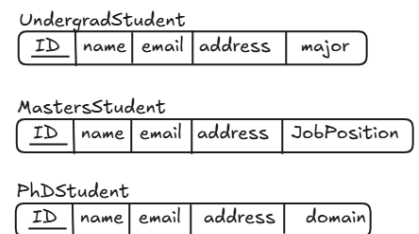
Specialization/Generalization Mapping



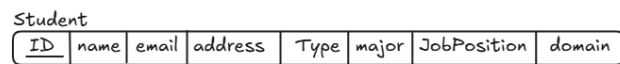
8A Mapping:



8B Mapping:



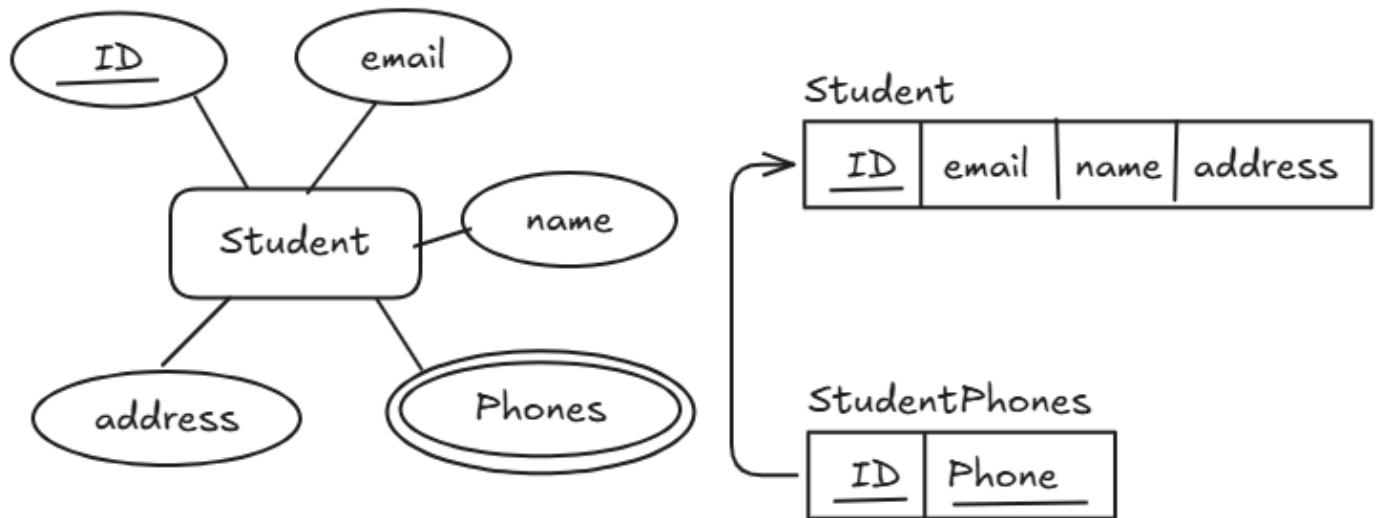
8C Mapping:



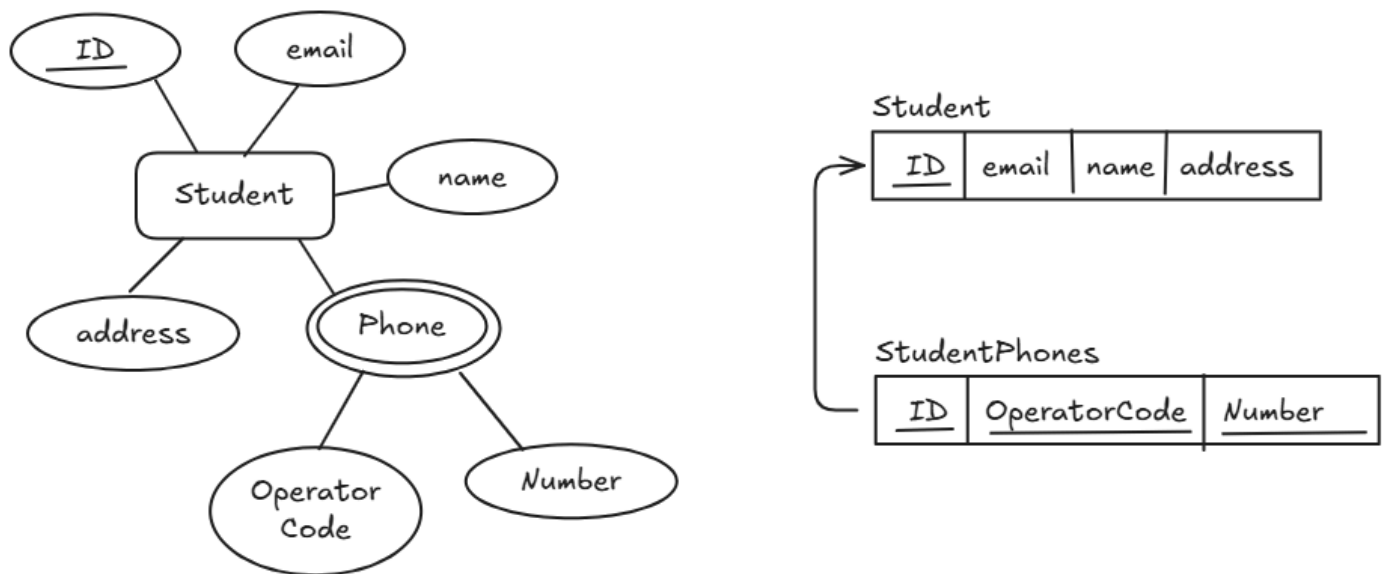
8D Mapping:



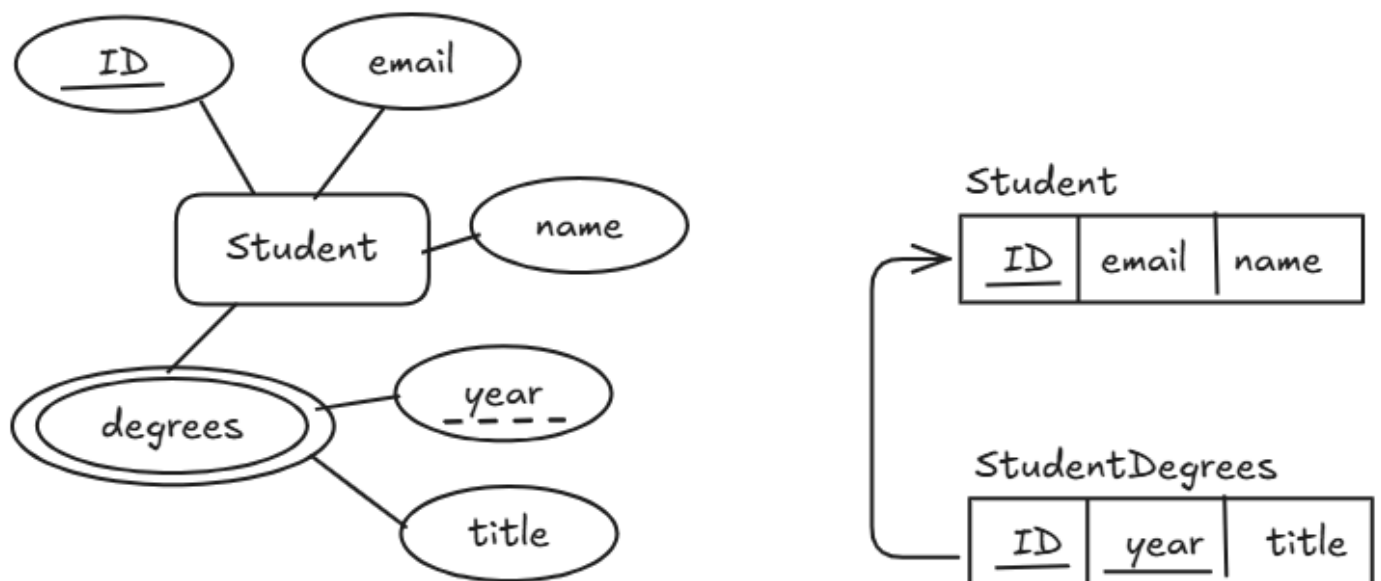
- Multivalued Attribute



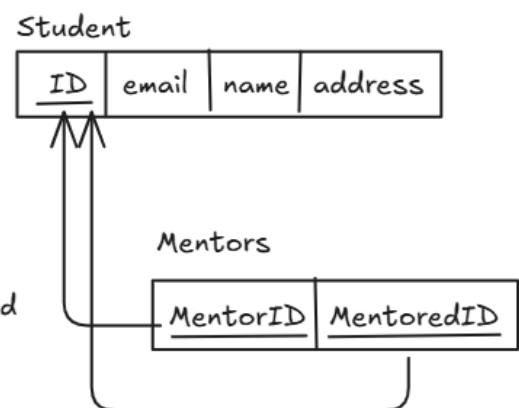
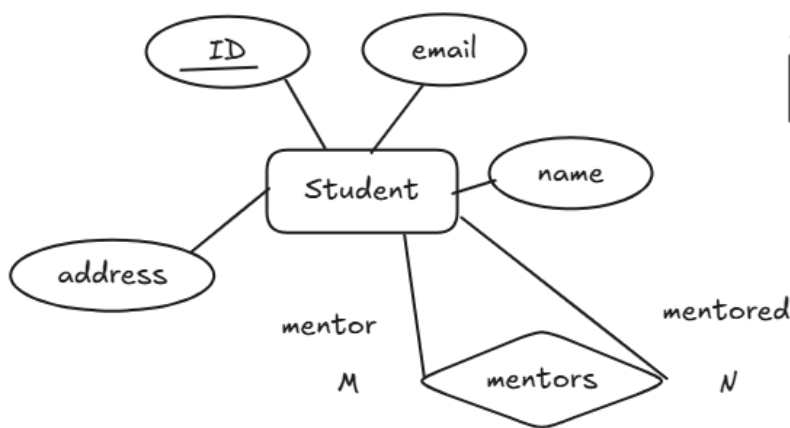
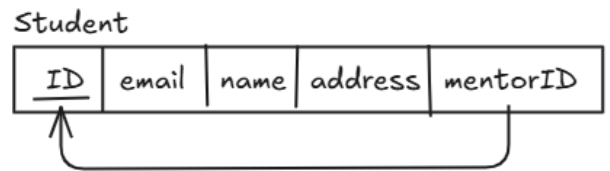
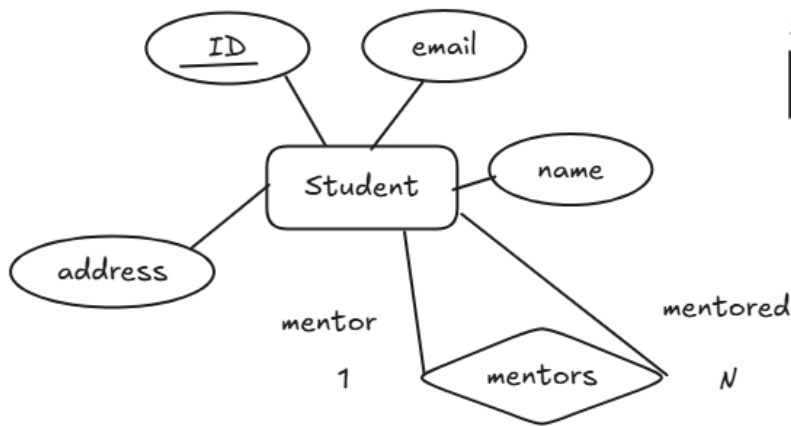
- Multivalued Composite Attribute



- Multivalued Composite Attribute with Partial attribute



- Recursive Relationship



Normalization

Things to keep in mind:

- Informal guidelines and Functional Dependencies (FD)
- 1NF disallows
 - composite attributes
 - multivalued attributes
 - nested relations
- 2NF
 - Must be in 1NF,
 - All attributes depend on the whole key. Disallows Partial Dependencies (transitive dependencies can exist in 2NF)
- 3NF
 - Must be in 2NF
 - All attributes depend on nothing but the key (No transitive dependencies exist)

Normalization_Example_Solved

Watchlist (streamerID, showName, SeasonNo, streamerName, subscriptionPlan, region, genre, releaseYear, totalEpisodes, producerPhone, producerName, producerEmail, platform, TRP, watchTimeLine, userRating)

The primary key is underlined.

The relation has the following additional functional dependencies (FDs):

FD1: streamerID → streamerName, subscriptionPlan, region, platform

FD2: showName → genre, producerName, producerPhone, producerEmail, releaseYear

FD3: producerName → producerPhone, producerEmail

FD4: showName, SeasonNo → TRP

FD5: subscriptionPlan, region → platform

- a. Explain if this is in **1NF** or not. If not, normalize it to 1NF. [2]
 - b. Explain if this is in **2NF** or not. If not, normalize it to 2NF. [4]
 - c. Explain if this is in **3NF** or not. If not, normalize it to 3NF. [4]
-

Solution:

Implicit /hidden FD: streamerID, showName, SeasonNo → totalEpisodes, watchTimeLine, userRating

- a. Already in 1NF due to no composite, multivalued attribute and no nested relation
- b. Not in 2NF due to partial FD of FD1, FD2, FD4

Decomposing to 2NF:

Streamer

(streamerID, streamerName, subscriptionPlan, region, platform)

Shows

(showName, genre, producerName, producerPhone, producerEmail, releaseYear)

Rating

(showName, SeasonNo, TRP)

Watchlist

(streamerID, showName, SeasonNo, totalEpisodes, watchTimeLine, userRating)

- c. Not in 3NF due to transitive dependency of FD3, FD5

Decomposing to 3NF:

Shows

(showName, genre, producerName, releaseYear)

Producer

(producerName, producerPhone, producerEmail)

Streamer

(streamerID, streamerName, subscriptionPlan, region)

Plans

(subscriptionPlan, region, platform)

Rating and Watchlist relation remains unchanged.

Relational Algebra

Things to keep in mind:

Symbol	Name	What it does	Example
σ	Selection	Filters rows matching a condition. Like WHERE in SQL	$\sigma_{\text{age} > 30} (\text{Patient})$
π	Projection	Picks specific columns, removes duplicates. Like SELECT in SQL	$\pi_{\text{name, age}} (\text{Patient})$
ρ	Rename	Renames a relation or its attributes.	$\rho_{\text{PatientInfo}}(\text{Patient})$
$\cup$	Union	All rows from both relations, duplicates removed.	$R \cup S$
$\cap$	Intersection	Only rows that appear in both relations.	$R \cap S$
$-$	Difference	Rows in R that do not appear in S.	$R - S$
$\times$	Cross product	Every row of R paired with every row of S.	$R \times S$
$\bowtie_{\theta}$	Theta join	Cross product filtered by condition θ .	$R \bowtie_{(\text{id}=\text{id})} S$

When Drawing RA Tree:

Draw from the bottom up, starting with base relations and building up to the final result.

Leaves: Each base relation (table) is a leaf node.

Unary Operations (Selection σ , Projection π , Rename ρ):

Place the operation as a parent node above the relation it applies to.

Example: $\sigma_{\text{condition}}(\text{Employee}) \rightarrow$ Draw $\sigma_{\text{condition}}$ above **Employee**.

Binary Operations (Join $\bowtie$, Cross Product $\times$, Union $\cup$, intersection $\cap$, Set Difference $-$):

Place the operation as a parent node with two children: the left and right operands.

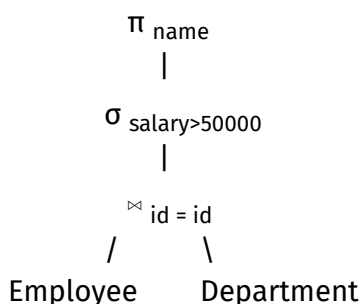
Example:

Employee $\bowtie_{\text{id} = \text{id}}$ **Department** $\rightarrow$ Draw $\bowtie_{\text{id} = \text{id}}$ above both **Employee** and **Department**.

Example:

Expression: $\pi_{\text{name}}(\sigma_{\text{salary} > 50000}(\text{Employee} \bowtie_{\text{id} = \text{id}} \text{Department}))$

Start from the bottom (right side) and build up (to the left side)

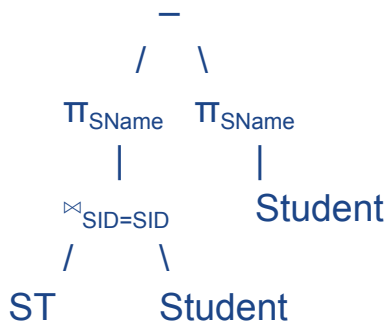


Some Question Types:

- Given the Schema, asked to write some RA
- Given a SQL query, asked to write equivalent RA
- Given a RA, draw the corresponding RA Tree

RA Tree Example:

$\pi_{SName}(\text{Student}) - \pi_{SName}(\text{Student} \bowtie_{SID=SID} ST)$



$\pi_{SID, SName, CName}((\text{Student} \bowtie_{SID=SID} ST) \bowtie_{AssignedCourse=CCode} \text{Course})$

